

计算机组成与体系结构课内实践 #1

提交时间: 23:59, 5 月 24 日, 2026

河海大学计算机与软件学院

声明: 在本次作业中请做到以下几点:

- 请将学号和姓名清晰的写在作业第一页, 请给出详细的注释, 严禁抄袭。
- 作业需提交源码、测试代码、运行结果和实验报告。请将所有提交材料打包为一个 zip 附件, 并通过电子邮件发送给任课教师。附件命名格式必须为” **学号-姓名-计组实践作业 1.zip**”。zip 附件中应包含 PDF 格式的实验报告、Verilog 源码文件和测试代码文件。运行结果应以截图形式放入 PDF 实验报告中, 并对关键仿真结果进行说明。请注意, 实验报告只接受 PDF 格式, 源码文件和测试代码文件必须为*.v文件, 压缩包只接受 zip 格式, 不接受其他压缩文件格式。
- 请确保电子邮件标题严格遵循以下格式: ” **学号-姓名-计组实践作业 1**”。若因邮件标题或附件命名不规范导致作业无法及时定位, 可能影响成绩记录。
- 本次作业不接受迟交, 若因客观原因无法完成, 请提前 24 小时联系老师并说明情况。

运算单元与存储系统综合设计

学习目标

- 巩固前五周所学运算器设计方法, 理解 ALU 操作码、标志位和组合逻辑运算;
- 理解寄存器堆的读写机制, 以及运算结果写回寄存器的过程;
- 理解数据存储器的读写控制、地址生成和 Load/Store 基本数据通路;
- 掌握运算单元、寄存器堆和存储器之间的数据连接关系;
- 能够编写 testbench 验证多周期操作序列中的数据变化。

题目一：16 位多功能 ALU 设计

请设计一个 16 位多功能算术逻辑单元 ALU。该 ALU 接收两个 16 位操作数，根据操作码完成不同运算，并输出运算结果及状态标志位。

模块接口如下：

```
1 module alu16 (  
2     input wire [15:0] a,  
3     input wire [15:0] b,  
4     input wire [2:0] alu_op,  
5     output reg [15:0] result,  
6     output wire zero,  
7     output wire carry,  
8     output wire negative,  
9     output wire overflow  
10 );
```

Listing 1: 16 位 ALU 模块接口

ALU 功能要求如下：

alu_op	功能	说明
3'b000	加法	result = a + b
3'b001	减法	result = a - b
3'b010	按位与	result = a & b
3'b011	按位或	result = a b
3'b100	按位异或	result = a ^ b
3'b101	逻辑左移	result = a << b[3:0]
3'b110	逻辑右移	result = a >> b[3:0]
3'b111	小于比较	若 a < b, 则 result = 16'h0001, 否则为 16'h0000

标志位要求如下：

- zero: 当 result == 16'h0000 时为 1，否则为 0；
- negative: 等于 result[15]；
- carry: 仅在加法和减法时有意义。加法时表示最高位进位，减法时表示无借位；
- overflow: 仅在有符号加法和有符号减法时有意义，其他操作输出 0；
- 其他逻辑运算、移位运算和比较运算中，carry 和 overflow 均输出 0。

设计要求如下：

- 使用 `always @(*)` 和 `case` 语句实现 ALU 主逻辑；
- 加法和减法需要考虑 `carry` 和 `overflow`；
- 比较操作按无符号数比较即可；
- 需要给出清晰注释，说明每一种操作码对应的功能；
- 不允许只实现加法、减法和逻辑与或，必须完整实现表中 8 种操作。

测试代码要求如下：

1. 测试普通加法，例如 `16'h0003 + 16'h0005`；
2. 测试产生进位的加法，例如 `16'hFFFF + 16'h0001`；
3. 测试有符号溢出的加法，例如 `16'h7FFF + 16'h0001`；
4. 测试普通减法，例如 `16'h0008 - 16'h0003`；
5. 测试结果为 0 的减法，验证 `zero` 标志；
6. 测试按位与、按位或、按位异或；
7. 测试逻辑左移和逻辑右移；
8. 测试小于比较功能。

报告要求如下：

- 解释 `zero`、`carry`、`negative` 和 `overflow` 四个标志位的含义；
- 说明有符号溢出和无符号进位的区别；
- 给出关键仿真波形，并说明每组测试是否符合预期。

题目二：ALU 与寄存器堆组合的数据通路设计

请在题目一 ALU 的基础上，设计一个包含寄存器堆和 ALU 的简化运算数据通路。该数据通路能够从寄存器堆中读取两个操作数，送入 ALU 进行运算，并将结果写回寄存器堆。

本题用于理解 CPU 中最基本的寄存器型运算过程：

读取源寄存器 → ALU 运算 → 写回目的寄存器

模块接口如下：

```
1 module alu_reg_datapath (  
2     input wire clk,  
3     input wire rst_n,  
4  
5     input wire reg_we,  
6     input wire [2:0] rs1,  
7     input wire [2:0] rs2,  
8     input wire [2:0] rd,  
9     input wire [2:0] alu_op,  
10  
11     input wire init_we,  
12     input wire [2:0] init_addr,  
13     input wire [15:0] init_data,  
14  
15     output wire [15:0] src_a,  
16     output wire [15:0] src_b,  
17     output wire [15:0] alu_result,  
18     output wire zero,  
19     output wire carry,  
20     output wire negative,  
21     output wire overflow  
22 );
```

Listing 2: ALU 与寄存器堆组合数据通路模块接口

内部结构要求如下：

- 内部包含一个 8x16 寄存器堆；
- 内部包含“题目一”设计的 alu16 模块；
- 寄存器堆包含 8 个 16 位寄存器，编号为 R0 到 R7；
- R0 规定为常零寄存器，读取时恒为 0，对 R0 的写入无效；

- rs1 和 rs2 分别指定两个源寄存器；
- rd 指定目的寄存器；
- src_a 和 src_b 分别为从寄存器堆读出的两个操作数；
- alu_result 为 ALU 输出结果；
- 当 reg_we = 1 时，在时钟上升沿将 alu_result 写入 rd 指定的寄存器；
- init_we 用于测试时初始化寄存器堆。当 init_we = 1 时，应将 init_data 写入 init_addr 指定的寄存器；
- 若 init_we 和 reg_we 同时为 1，规定 init_we 优先。

寄存器堆接口建议如下：

```

1 module regfile8x16 (
2     input wire clk,
3     input wire rst_n,
4     input wire we,
5     input wire [2:0] waddr,
6     input wire [15:0] wdata,
7     input wire [2:0] raddr1,
8     input wire [2:0] raddr2,
9     output wire [15:0] rdata1,
10    output wire [15:0] rdata2
11 );

```

Listing 3: 8x16 寄存器堆模块接口

寄存器堆设计要求如下：

- 写操作在时钟上升沿完成；
- 读操作为组合逻辑读出；
- 复位时所有寄存器清零；
- R0 恒为 0，不能被写入；
- 当同一周期内写入地址与读地址相同，并且写使能有效时，读端口应输出即将写入的新数据，该逻辑称为写后读旁路。

测试要求如下：

请在 testbench 中至少完成如下操作序列：

1. 复位系统，验证所有寄存器初值为 0；
2. 使用 `init_we` 初始化：
 - `R1 = 16'h0005`
 - `R2 = 16'h0003`
 - `R3 = 16'h00F0`
 - `R4 = 16'h0F0F`
3. 执行 `R5 = R1 + R2`，期望 `R5 = 16'h0008`；
4. 执行 `R6 = R5 - R2`，期望 `R6 = 16'h0005`；
5. 执行 `R7 = R3 & R4`，期望 `R7 = 16'h0000`，同时验证 `zero` 标志；
6. 尝试执行 `R0 = R1 + R2`，再读取 `R0`，验证其仍为 0；
7. 构造同周期读写同一寄存器的情况，验证旁路逻辑是否正确。

报告要求如下：

- 画出本题的数据通路图；
- 用表格列出每一步操作对应的 `rs1`、`rs2`、`rd`、`alu_op` 和 `reg_we`；
- 说明寄存器堆、ALU 和写回路之间的数据关系；
- 解释为什么需要 `init_we`；
- 解释写后读旁路的作用；
- 给出仿真波形并标注关键周期。

题目三：支持 Load/Store 的简化运算与存储数据通路设计

请在题目二的基础上继续扩展，加入数据存储器，实现一个支持 ALU 运算、Load 和 Store 操作的简化数据通路。该模块不要求实现完整指令译码器，但需要通过控制信号完成不同类型的数据传输。

本题是本次作业的综合题。目标是理解如下三类操作：

- 寄存器型运算：rd = rs1 op rs2
- Store 操作：Mem[rs1 + imm] = rs2
- Load 操作：rd = Mem[rs1 + imm]

模块接口如下：

```
1 module ls_datapath (  
2     input wire clk,  
3     input wire rst_n,  
4  
5     input wire reg_we,  
6     input wire mem_we,  
7     input wire mem_to_reg,  
8     input wire alu_src_imm,  
9  
10    input wire [2:0] rs1,  
11    input wire [2:0] rs2,  
12    input wire [2:0] rd,  
13    input wire [15:0] imm,  
14    input wire [2:0] alu_op,  
15    input wire [1:0] byte_en,  
16  
17    input wire init_we,  
18    input wire [2:0] init_addr,  
19    input wire [15:0] init_data,  
20  
21    output wire [15:0] src_a,  
22    output wire [15:0] src_b,  
23    output wire [15:0] alu_result,  
24    output wire [15:0] mem_rdata,  
25    output wire [15:0] wb_data,  
26    output wire zero  
27 );
```

Listing 4: 支持 Load/Store 的简化数据通路模块接口

内部模块要求如下：

- 内部包含题目一的 alu16;
- 内部包含题目二中的 regfile8x16;
- 内部包含一个 32x16 数据存储器;
- 数据存储器地址使用 alu_result[4:0];
- 数据存储器写入数据来自 src_b;
- 写回寄存器堆的数据由 mem_to_reg 选择:
 1. 当 mem_to_reg = 0 时, 写回 ALU 结果;
 2. 当 mem_to_reg = 1 时, 写回数据存储器读出的数据。
- ALU 第二操作数由 alu_src_imm 选择:
 1. 当 alu_src_imm = 0 时, 第二操作数为 src_b;
 2. 当 alu_src_imm = 1 时, 第二操作数为 imm。

数据存储器接口建议如下：

```
1 module data_mem32x16 (  
2     input wire clk,  
3     input wire mem_we,  
4     input wire [4:0] addr,  
5     input wire [15:0] wdata,  
6     input wire [1:0] byte_en,  
7     output reg [15:0] rdata  
8 );
```

Listing 5: 32x16 数据存储器模块接口

数据存储器设计要求如下：

- 使用 reg [15:0] mem [0:31]; 定义 32 个 16 位存储单元;
- 写操作在时钟上升沿完成;

- 读操作采用组合逻辑读出；
- 当 `byte_en = 2'b11` 时写入完整 16 位数据；
- 当 `byte_en = 2'b01` 时只写入低 8 位；
- 当 `byte_en = 2'b10` 时只写入高 8 位；
- 当 `byte_en = 2'b00` 时不修改存储器内容；
- 可在 `initial` 块中将存储器初始化为 0，便于仿真。

三类操作的控制信号建议如下：

操作类型	reg_we	mem_we	mem_to_reg	alu_src_imm	alu_op
ALU 运算	1	0	0	0	由运算类型决定
Load	1	0	1	1	加法
Store	0	1	任意	1	加法

测试要求如下：

请在 `testbench` 中至少完成以下操作序列：

1. 复位系统；
2. 使用 `init_we` 初始化寄存器：
 - `R1 = 16'h0004`
 - `R2 = 16'h0006`
 - `R3 = 16'hABCD`
3. 执行寄存器型 ALU 运算：`R4 = R1 + R2`，期望 `R4 = 16'h000A`；
4. 执行寄存器型 ALU 运算：`R5 = R4 - R1`，期望 `R5 = 16'h0006`；
5. 执行 Store 操作：将 `R3` 写入地址 `R1 + 0`，即数据存储器地址 `5'd4`；
6. 执行 Load 操作：从地址 `R1 + 0` 读出数据并写入 `R6`，期望 `R6 = 16'hABCD`；
7. 执行低字节 Store：将某个寄存器的低字节写入地址 `R1 + 1`，验证 `byte_en = 2'b01` 是否生效；
8. 执行高字节 Store：将某个寄存器的高字节写入地址 `R1 + 1`，验证 `byte_en = 2'b10` 是否生效；

9. 尝试写入 R0, 验证 R0 仍为 0;
10. 至少构造一次 ALU 结果为 0 的操作, 验证 zero 标志。

报告要求如下:

- 画出完整数据通路图 (请用 Microsoft Visio 或者 Powerpoint 画图), 包括寄存器堆、ALU、数据存储器 and 写回多路选择器;
- 用表格列出每一步测试操作的控制信号, 包括 reg_we、mem_we、mem_to_reg、alu_src_imm、alu_op 和 byte_en;
- 解释 Load 和 Store 操作中 ALU 为什么用于地址计算;
- 说明 mem_to_reg 和 alu_src_imm 两个多路选择控制信号的作用;
- 解释字节写使能在存储系统中的意义;
- 根据仿真波形说明数据如何从寄存器堆进入 ALU、如何写入存储器, 以及如何从存储器读出并写回寄存器堆。

提交材料说明

- Verilog 代码文件 (.v);
- Testbench 代码文件 (.v);
- 仿真波形截图, 如 ModelSim、Vivado、Quartus;
- 实验报告 PDF, 包含设计说明、模块接口说明、测试截图、关键代码解释和仿真结果分析;
- 每一道题均需说明模块功能、输入输出端口含义、测试用例设计和仿真结果是否符合预期。

评分标准

- 题目一: 30 分;
- 题目二: 30 分;
- 题目三: 40 分;

题目三详细说明与解题步骤

支持 Load/Store 的简化运算与存储数据通路

题目三要求设计的是一个支持运算、Load 和 Store 的简化数据通路。它不是一个完整 CPU，因为它没有程序计数器、指令存储器、指令译码器和控制器；但是它已经包含了 CPU 执行指令时最核心的三个数据部件：**寄存器堆、ALU 和数据存储器**。

你们可以把这个模块理解为一个“手动控制版的 CPU 执行部件”。外部控制信号相当于已经被译码好的控制命令。例如，`reg_we` 控制是否写回寄存器，`mem_we` 控制是否写内存，`mem_to_reg` 控制写回寄存器的数据来自 ALU 还是内存，`alu_src_imm` 控制 ALU 的第二个操作数来自寄存器还是立即数。你们不需要设计指令译码器，只需要根据这些控制信号，把数据正确地送到对应部件。

本题最终要实现三类基本操作。第一类是普通寄存器运算，例如 $R4 = R1 + R2$ ，数据从寄存器堆读出，经过 ALU 运算后再写回寄存器堆。第二类是 Store 操作，例如 $Mem[R1 + 0] = R3$ ，其中 ALU 用来计算内存地址 $R1 + 0$ ，真正写入内存的数据来自 R3。第三类是 Load 操作，例如 $R6 = Mem[R1 + 0]$ ，其中 ALU 仍然用来计算内存地址，而最终写回 R6 的数据来自数据存储器。

所以，这道题考察的不是某一个孤立模块，而是考察你们是否理解：**寄存器堆如何提供操作数，ALU 如何完成运算或地址计算，数据存储器如何读写，以及写回通路如何选择正确的数据来源**。这正是后续理解单周期 CPU 数据通路的基础。

二、解题步骤

第一步：先完成并验证 ALU 模块

先不要急着写顶层模块。首先确认题目一中的 `alu16` 能够单独工作。

ALU 至少要支持以下功能：

加法、减法、按位与、按位或、按位异或、左移、右移、比较

在题目三中，ALU 有两个作用：

- 普通运算时：计算最终结果，例如 $R4 = R1 + R2$ ；
- Load/Store 时：计算内存地址，例如 $R1 + imm$ 。

因此你要明确一点：**ALU 的输出不一定总是要写回寄存器**。有时候它只是一个内存地址。建议先用 `testbench` 单独测试：

```
1 16'h0004 + 16'h0006 = 16'h000A
2 16'h000A - 16'h0004 = 16'h0006
3 16'h00F0 & 16'h0F0F = 16'h0000
```

Listing 6: ALU 测试示例

确认 ALU 正确后，再进入下一步。

第二步：完成并验证寄存器堆模块

寄存器堆可以理解为一个小型寄存器数组，里面有 8 个 16 位寄存器：

R0, R1, R2, R3, R4, R5, R6, R7

它需要支持：

- 两个读端口：同时读出 `rs1` 和 `rs2` 指定的寄存器；
- 一个写端口：把结果写入 `rd` 指定的寄存器。

其中 R0 要设计成常零寄存器：

- 读取 R0 时永远得到 16'h0000；
- 写入 R0 时应当无效。

寄存器堆的基本行为是：

```
1 rdata1 = Reg[rs1]
2 rdata2 = Reg[rs2]
3
4 if write enable is valid:
5     at the rising edge of clk:
6         Reg[rd] = write_back_data
```

Listing 7: 寄存器堆基本行为

注意：写入通常是同步的，也就是必须等到时钟上升沿才生效；读出通常是组合逻辑，也就是读地址变化后，读数据会随之变化。

建议单独测试：

```

1 Initialize R1 = 16'h0004
2 Initialize R2 = 16'h0006
3 Read R1 and R2, and check whether the outputs are correct
4 Try to write R0, and check whether R0 is still 0

```

Listing 8: 寄存器堆测试示例

第三步：完成并验证数据存储器模块

数据存储器可以理解为一个简单 RAM。本题建议实现 32 个 16 位存储单元：

Mem[0] 到 Mem[31]

地址是 5 位，因为 5 位二进制可以表示 32 个地址。

数据存储器需要支持：

- 读操作：根据地址读出 Mem[addr]；
- 写操作：在 mem_we = 1 时，把数据写入 Mem[addr]；
- 字节写使能：byte_en 控制写高字节、低字节或完整 16 位。

其中 byte_en 的含义是：

byte_en	含义
2'b11	写完整 16 位
2'b01	只写低 8 位
2'b10	只写高 8 位
2'b00	不写

建议单独测试：

```

1 Write 16'hABCD to Mem[4], then read it back.
2 Expected result: 16'hABCD
3
4 Write only the low byte 16'h00EF to Mem[4].
5 Expected result: 16'hABEF
6
7 Write only the high byte 16'h1200 to Mem[4].
8 Expected result: 16'h12EF

```

Listing 9: 数据存储器测试示例

第四步：在顶层模块中连接三个子模块

当 ALU、寄存器堆和数据存储器都能单独工作后，再写顶层模块 `ls_datapath`。

顶层模块要完成的不是复杂计算，而是**正确连线**和**正确选择数据来源**。

你需要在顶层模块里准备几类内部信号：

```
1 wire [15:0] src_a;
2 wire [15:0] src_b;
3 wire [15:0] alu_b;
4 wire [15:0] alu_result;
5 wire [15:0] mem_rdata;
6 wire [15:0] wb_data;
7 wire [4:0] mem_addr;
```

Listing 10: 顶层模块内部信号

它们的含义分别是：

信号	含义
<code>src_a</code>	从 <code>rs1</code> 读出的寄存器数据
<code>src_b</code>	从 <code>rs2</code> 读出的寄存器数据
<code>alu_b</code>	ALU 的第二个操作数
<code>alu_result</code>	ALU 计算结果
<code>mem_rdata</code>	数据存储器读出的数据
<code>wb_data</code>	最终写回寄存器堆的数据
<code>mem_addr</code>	数据存储器地址

顶层模块中最关键的是三个选择关系。

第五步：实现 ALU 第二操作数选择

ALU 的第一个操作数固定来自 `src_a`。

ALU 的第二个操作数有两种来源：

- 普通 ALU 运算时，来自 `src_b`；
- Load/Store 计算地址时，来自 `imm`。

因此可以这样理解：

```
1 assign alu_b = alu_src_imm ? imm : src_b;
```

Listing 11: ALU 第二操作数选择

当 `alu_src_imm = 0` 时:

ALU 计算 `src_a op src_b`, 例如 `R4 = R1 + R2`。

当 `alu_src_imm = 1` 时:

ALU 计算 `src_a op imm`, 例如地址 `R1 + 0`。

第六步：实现数据存储器地址选择

数据存储器只有 32 个单元，因此地址只需要 5 位。

ALU 输出是 16 位，所以题目规定使用低 5 位作为存储器地址：

```
1 assign mem_addr = alu_result[4:0];
```

Listing 12: 数据存储器地址选择

这表示：

ALU 计算出的结果作为访存地址，但数据存储器只使用其中低 5 位。

例如：

```
1 R1 = 16'h0004
2 imm = 16'h0000
3 alu_result = 16'h0004
4 mem_addr = 5'd4
```

Listing 13: 地址计算示例

第七步：实现写回数据选择

写回寄存器的数据有两种来源：

- 普通 ALU 运算：写回 ALU 结果；
- Load 操作：写回数据存储器读出的数据。

因此需要一个写回多路选择器：

```
1 assign wb_data = mem_to_reg ? mem_rdata : alu_result;
```

Listing 14: 写回数据选择

当 `mem_to_reg = 0` 时:

```
wb_data = alu_result
```

当 `mem_to_reg = 1` 时:

```
wb_data = mem_rdata
```

这是 Load 操作能否正确完成的关键。如果 Load 时忘记把 `mem_to_reg` 设为 1，就会把地址写回寄存器，而不是把内存数据写回寄存器。

第八步：处理寄存器初始化写入

题目中有 `init_we`、`init_addr`、`init_data`，目的是让 testbench 可以先给寄存器堆写入初始值。

因为正常运行时写回寄存器的是 `rd` 和 `wb_data`，初始化时写入的是 `init_addr` 和 `init_data`，所以需要选择真正送入寄存器堆写端口的地址和数据。

可以按这个逻辑理解：

```
1 if init_we = 1:
2     write init_data to init_addr
3 else:
4     normal write-back, write wb_data to rd
```

Listing 15: 初始化写入逻辑

对应关系可以写成：

```
1 assign real_reg_we = init_we ? 1'b1 : reg_we;
2 assign real_waddr  = init_we ? init_addr : rd;
3 assign real_wdata  = init_we ? init_data : wb_data;
```

Listing 16: 寄存器写端口选择

这里 `init_we` 的优先级高于 `reg_we`。也就是说，只要处于初始化阶段，就先完成寄存器初始化，不执行正常写回。

第九步：理解三类操作的控制信号

1. 普通 ALU 运算

例如：

$$R4 = R1 + R2$$

控制信号应当是：

控制信号	取值
reg_we	1
mem_we	0
mem_to_reg	0
alu_src_imm	0
alu_op	加法

解释如下：

- 需要写回寄存器，所以 `reg_we = 1`；
- 不写内存，所以 `mem_we = 0`；
- 写回数据来自 ALU，所以 `mem_to_reg = 0`；
- ALU 第二操作数来自 `rs2`，所以 `alu_src_imm = 0`。

2. Store 操作

例如：

$$\text{Mem}[R1 + 0] = R3$$

控制信号应当是：

控制信号	取值
reg_we	0
mem_we	1
mem_to_reg	任意
alu_src_imm	1
alu_op	加法
byte_en	2'b11

解释如下：

- Store 不写寄存器，所以 `reg_we = 0`；

- Store 要写内存，所以 `mem_we = 1`;
- ALU 用来计算地址 `R1 + imm`，所以 `alu_src_imm = 1`，`alu_op` 为加法;
- 写入内存的数据来自 `src_b`，也就是 `rs2` 读出的值。

尤其注意：

Store 中，`alu_result` 是地址，不是写入内存的数据。

3. Load 操作

例如：

`R6 = Mem[R1 + 0]`

控制信号应当是：

控制信号	取值
<code>reg_we</code>	1
<code>mem_we</code>	0
<code>mem_to_reg</code>	1
<code>alu_src_imm</code>	1
<code>alu_op</code>	加法

解释如下：

- Load 要把内存数据写回寄存器，所以 `reg_we = 1`;
- Load 不写内存，所以 `mem_we = 0`;
- 写回数据来自内存，所以 `mem_to_reg = 1`;
- ALU 用来计算地址 `R1 + imm`，所以 `alu_src_imm = 1`，`alu_op` 为加法。

尤其注意：

Load 中，`alu_result` 是地址，真正写回 `rd` 的是 `mem_rdata`。

第十步：按固定顺序写 testbench

建议 testbench 按下面顺序写，不要随机给输入。

1. 复位系统

先让 `rst_n = 0`，保持若干时间，然后拉高：

```
1 rst_n = 0;
2 wait for two clock cycles;
3 rst_n = 1;
```

Listing 17: 复位系统

检查寄存器是否清零。

2. 初始化寄存器

用 `init_we` 初始化：

寄存器	初始值
R1	16'h0004
R2	16'h0006
R3	16'hABCD

每写一个寄存器都要等待一个时钟上升沿。

3. 执行普通 ALU 运算

执行：

$$R4 = R1 + R2$$

期望：

$$R4 = 16'h000A$$

控制信号：

信号	取值
<code>rs1</code>	R1
<code>rs2</code>	R2
<code>rd</code>	R4
<code>alu_op</code>	加法
<code>alu_src_imm</code>	0
<code>mem_to_reg</code>	0
<code>mem_we</code>	0
<code>reg_we</code>	1

4. 执行 Store

执行:

$$\text{Mem}[\text{R1} + 0] = \text{R3}$$

因为:

$$\text{R1} = 16'h0004, \quad \text{imm} = 16'h0000$$

所以实际写入地址为:

$$\text{Mem}[4]$$

期望:

$$\text{Mem}[4] = 16'hABCD$$

控制信号:

信号	取值
rs1	R1
rs2	R3
imm	0
alu_op	加法
alu_src_imm	1
mem_we	1
reg_we	0
byte_en	2'b11

5. 执行 Load

执行:

$$\text{R6} = \text{Mem}[\text{R1} + 0]$$

期望:

$$\text{R6} = 16'hABCD$$

控制信号:

信号	取值
rs1	R1
rd	R6
imm	0
alu_op	加法
alu_src_imm	1
mem_we	0
mem_to_reg	1
reg_we	1

6. 测试字节写使能

可以在同一个内存地址上测试低字节写和高字节写。

例如，某地址原值为：

```
Mem[5] = 16'h1234
```

低字节写：

```
写入数据 = 16'h00AA, byte_en = 2'b01
```

预期：

```
Mem[5] = 16'h12AA
```

高字节写：

```
写入数据 = 16'hBB00, byte_en = 2'b10
```

预期：

```
Mem[5] = 16'hBBAA
```

三、最后检查清单

学生完成后至少检查以下几点：

1. ALU 第二操作数是否由 alu_src_imm 正确选择？

2. 写回数据是否由 `mem_to_reg` 正确选择?
3. Store 时, 写入内存的数据是否来自 `src_b`?
4. Load 时, 写回寄存器的数据是否来自 `mem_rdata`?
5. 数据存储器地址是否使用 `alu_result[4:0]`?
6. `init_we` 是否优先于正常 `reg_we`?
7. R0 是否始终保持为 0?
8. `byte_en = 2'b01` 时是否只修改低 8 位?
9. `byte_en = 2'b10` 时是否只修改高 8 位?
10. `testbench` 是否在时钟上升沿之后再检查写入结果?

四、总结

如果普通 ALU 运算、Store 和 Load 三类操作都能按预期完成, 这道题的核心功能就基本正确了。

本题的重点不是写出某一个单独的 Verilog 模块, 而是理解 CPU 数据通路中的数据流向: **寄存器堆提供操作数, ALU 完成运算或地址计算, 数据存储器完成读写, 写回通路决定最终写回寄存器的数据来源。**

这就是单周期 CPU 数据通路中最核心的一部分。